

Universidad Mayor de San Andrés



PROGRAMACION WEB III

Práctica1

Nombre Completo: Henry Dario Chipana Ergueta

CI:13083524 L.P.

Carrera: Informática

Materia: INF- 133

Fecha: 30/03/2026

1. Crear una función que cuente cuántas veces aparece cada vocal en un texto y devuelva el resultado en un objeto.

```
let obj = miFuncion("euforia")
```

```
console.log(obj)//{a:1,e:1,i:1,o:1,u:1}
```

```
const contarVocales = (palabra) => {  
  let resultado = {};  
  let listaVocales = "aeiou";  
  
  for (let i = 0; i < listaVocales.length; i++) {  
  
    let contador = 0;  
  
    for (let j = 0; j < palabra.length; j++) {  
      if (listaVocales[i] === palabra[j].toLowerCase()) {  
        contador++;  
      }  
    }  
  
    if (contador > 0) {  
      resultado[listaVocales[i]] = contador;  
    }  
  }  
  
  return resultado;  
}
```

```
let salida = contarVocales("Leonardo");  
console.log(salida);
```

```
{ a: 1, e: 1, o: 2 }
```

```
=== Code Execution Successful ===
```

2. Crear una función que invierta el orden de las palabras en una frase.

```
let cad=miFuncion("abcd")
```

```
console.log(obj) // dcba
```

```
const invertirTexto = (palabra) => {  
  
  let resultado = "";  
  
  for (let i = 0; i < palabra.length; i++) {  
    resultado = palabra[i] + resultado;  
  }  
  
  return resultado;  
}  
  
let textoInvertido = invertirTexto("holalic");  
console.log(textoInvertido);
```

cilaloh

=== Code Execution Successful ===

3. Crear una función que reciba un arreglo de números y devuelva en un objeto a los pares e impares:

```
const separarNumeros = (lista) => {  
  
  let resultado = {  
    pares: [],  
    impares: []  
  };  
  
  for (let i = 0; i < lista.length; i++) {  
    if (lista[i] % 2 === 0) {  
      resultado.pares.push(lista[i]);  
    } else {  
      resultado.impares.push(lista[i]);  
    }  
  }  
  
  return resultado;  
}  
  
let salida = separarNumeros([1, 2, 3, 4, 5]);  
console.log(salida);
```

```
{ pares: [ 2, 4 ], impares: [ 1, 3, 5 ] }
```

```
=== Code Execution Successful ===
```

4. Crear una función que reciba un arreglo de números y devuelva el número mayor y el menor, en un objeto

```
const obtenerMayorYMenor = (lista) => {  
  let resultado = {};
```

```
  let maximo = lista[0];  
  let minimo = lista[0];
```

```
  for (let i = 1; i < lista.length; i++) {  
    if (lista[i] > maximo) {  
      maximo = lista[i];  
    } else if (lista[i] < minimo) {  
      minimo = lista[i];  
    }  
  }
```

```
  resultado["maximo"] = maximo;  
  resultado["minimo"] = minimo;
```

```
  return resultado;  
};
```

```
let resultadoFinal = obtenerMayorYMenor([3, 1, 5, 4, 2]); console.log(resultadoFinal);
```

```
{ maximo: 5, minimo: 1 }
```

```
=== Code Execution Successful ===
```

5. Crear una función que determine si una cadena es palíndromo (se lee igual al derecho y al revé)

```
const verificarPalindromo = (texto) => {  
  let textoInvertido = "";  
  for (let i = 0; i < texto.length; i++) {  
    textoInvertido = texto[i] + textoInvertido;  
  }  
  return textoInvertido === texto;  
};  
  
let resultado1 = verificarPalindromo("oruro");  
console.log(resultado1);  
  
let resultado2 = verificarPalindromo("hola");  
console.log(resultado2);
```

```
true  
false  
  
=== Code Execution Successful ===
```

6. Tomar los dos primeros elementos de un arreglo y almacenarlos en dos variables mediante desestructuración.

```
const valores = [5, 8, 12, 20];  
  
const [valor1, valor2] = valores;  
  
console.log(valor1);  
console.log(valor2);
```

```
5  
8  
  
=== Code Execution Successful ===
```

7-. Almacenar el resto de los elementos de un arreglo sin tomar en cuenta los dos primeros elementos de un arreglo, mediante desestructuración.

```
let numeros = [10, 20, 30, 40, 50];  
let [primero, segundo, ...otros] = numeros;  
  
console.log(otros);
```

```
[ 30, 40, 50 ]  
  
=== Code Execution Successful ===
```

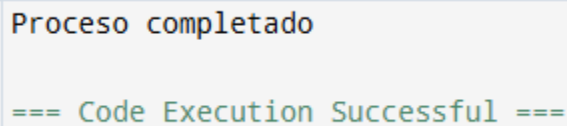
CallbackPromesasenJS

8-. Realizar un código para ejecutar una función callback después de 2 segundos

```
const tareaAsincrona = new Promise((terminar) => {
  setTimeout(() => {
    const resultado = ejecutarFuncion();
    terminar(resultado);
  }, 1500);
});

const ejecutarFuncion = () => {
  return "Proceso completado ";
};

tareaAsincrona.then((respuesta) => {
  console.log(respuesta);
});
```



```
Proceso completado

=== Code Execution Successful ===
```

9-. Crear una promesa que devuelva un mensaje de éxito después de 3 segundos.

```
const procesoAsync = new Promise((completar) => {
  setTimeout(() => {
    completar("Operación realizada con éxito");
  }, 2500);
});

procesoAsync
  .then((resultadoFinal) => {
    console.log(resultadoFinal);
  })
  .finally(() => {
    console.log("El flujo continúa...");
  });
```

```
Operación realizada con éxito  
El flujo continúa...
```

```
=== Code Execution Successful ===
```

10-. ¿Cuándo es conveniente utilizar un callback, y cuando es necesario utilizar una promesa?

- Debemos usar **callbacks** cuando queremos añadir alguna acción extra que se ejecute inmediatamente dentro de una función principal.
- De forma parecida, usamos **promesas** cuando necesitamos realizar algo después de cierto tiempo o cuando termina otra tarea . Es decir, primero se ejecuta la función principal y, si todo sale bien, se cumple la promesa; si ocurre un error, la promesa también lo maneja

11-.Proporcione un ejemplo concreto de encadenamiento de promesas.

```
const obtenerNumero = () => {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve(5), 1000);  
  });  
};  
obtenerNumero()  
  .then((num) => {  
    console.log("Primer valor:", num);  
    return num * 2;  
  })  
  .then((resultado) => {  
    console.log("Multiplicado:", resultado);  
    return resultado + 10;  
  })  
  .then((final) => {  
    console.log("Resultado final:", final);  
  })  
  .catch((error) => {  
    console.log("Ocurrió un error:", error);  
  });
```

```
Primer valor: 5  
Multiplicado: 10  
Resultado final: 20
```

```
=== Code Execution Successful ===
```


12-. Proporcione un ejemplo concreto donde el anidamiento de callbacks se puede reescribir mejor con async / await haciendo el código más limpio y mantenible.

```
const paso1 = () =>
  new Promise((resolve) =>
    setTimeout(() => resolve("Paso 1"), 1000)
  );

const paso2 = (dato) =>
  new Promise((resolve) =>
    setTimeout(() => resolve(dato + " -> Paso 2"), 1000)
  );

const paso3 = (dato) =>
  new Promise((resolve) =>
    setTimeout(() => resolve(dato + " -> Paso 3"), 1000)
  );

async function ejecutarProceso() {
  try {
    const r1 = await paso1();
    const r2 = await paso2(r1);
    const r3 = await paso3(r2);

    console.log(r3);
  } catch (error) {
    console.log("Error:", error);
  }
}

ejecutarProceso();
```

13-.Proporcione un ejemplo concreto donde el anidamiento de promesas se puede reescribir mejor con async/await haciendo el código más limpio y mantenible

```
async function ejecutarFlujo() {
  try {
    const datos = await obtenerDatos();
    const procesados = await procesarDatos(datos);
    const guardados = await guardarDatos(procesados);

    console.log(guardados);
  } catch (error) {
    console.log("Error:", error);
  }
}

ejecutarFlujo();
```

14-.Proporcione un ejemplo para convertir una promesa en un callback.

ORIGINAL

```
function obtenerUsuario() {
  return new Promise(resolve => setTimeout(() => resolve("Usuario obtenido"),
  1000));
}
```

CALLBACK.

```
function obtenerUsuarioCallback(callback) {
  setTimeout(() => {
    callback(null, "Usuario obtenido");
  }, 1000);
}

obtenerUsuarioCallback((error, resultado) => {
  if (error) console.log("Error:", error);
  else console.log(resultado);
});
```

15-.Proporcione un ejemplo para convertir un callback en una promesa.

CALLBACK

```
function obtenerDatosCallback(callback) {  
  setTimeout(() => {  
    // Primer parámetro es error, segundo resultado  
    callback(null, "Datos obtenidos");  
  }, 1000);  
}
```

PROMESA

```
function obtenerDatosPromesa() {  
  return new Promise((resolve, reject) => {  
    obtenerDatosCallback((error, resultado) => {  
      if (error) {  
        reject(error);  
      } else {  
        resolve(resultado);  
      }  
    });  
  });  
}
```

16-.Proporcione un ejemplo para migrar una función con promesas aasync/await.

ORIGINAL CON PROMESAS

```
function obtenerNumero() {
  return new Promise((resolve) => {
    setTimeout(() => resolve(10), 1000);
  });
}

function procesarNumero(num) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(num * 3), 1000);
  });
}

function sumarCinco(num) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(num + 5), 1000);
  });
}

// Uso con promesas encadenadas
obtenerNumero()
  .then((num) => procesarNumero(num))
  .then((res) => sumarCinco(res))
  .then((final) => console.log("Resultado final:", final))
  .catch((err) => console.log("Error:", err));
```

MIGRADO A ASYNC/AWAIT

```
async function ejecutarFlujo() {
  try {
    const num = await obtenerNumero();
    const procesado = await procesarNumero(num);
    const resultadoFinal = await sumarCinco(procesado);

    console.log("Resultado final:", resultadoFinal);
  } catch (error) {
    console.log("Error:", error);
  }
}

ejecutarFlujo();
```